电子线路设计、测试与实验

华中科技大学电子信息与通信学院
左冬红

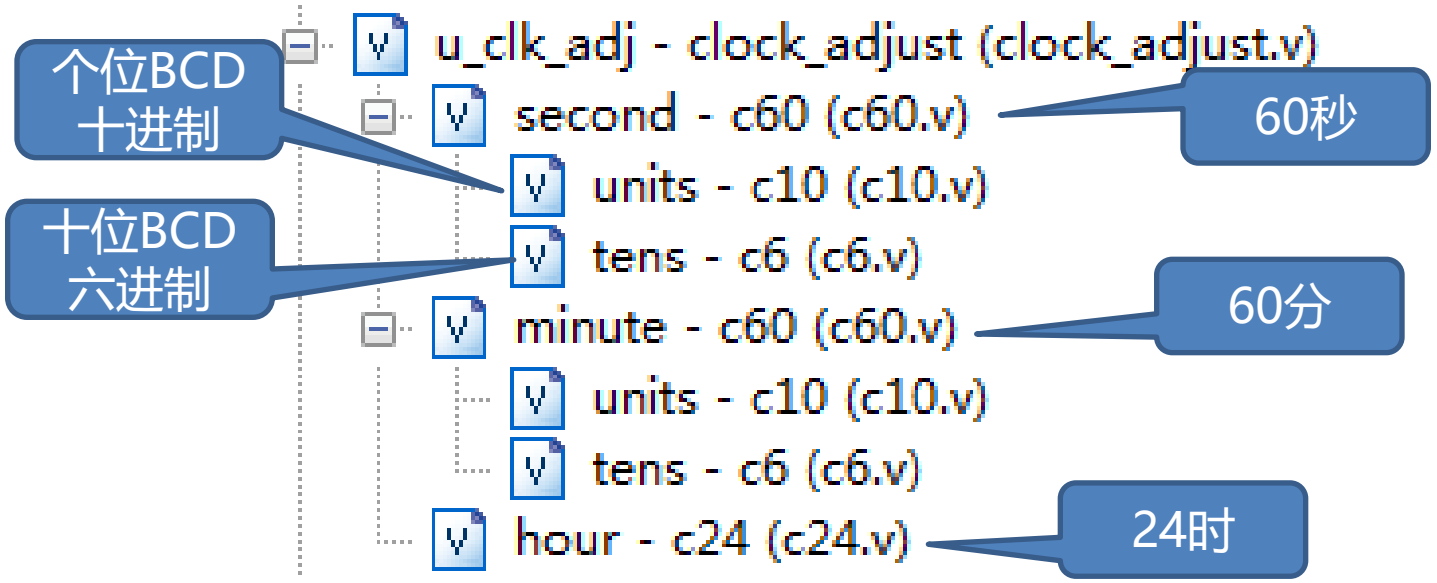
2 基于FPGA和EDA技术的多功能数字钟

实现

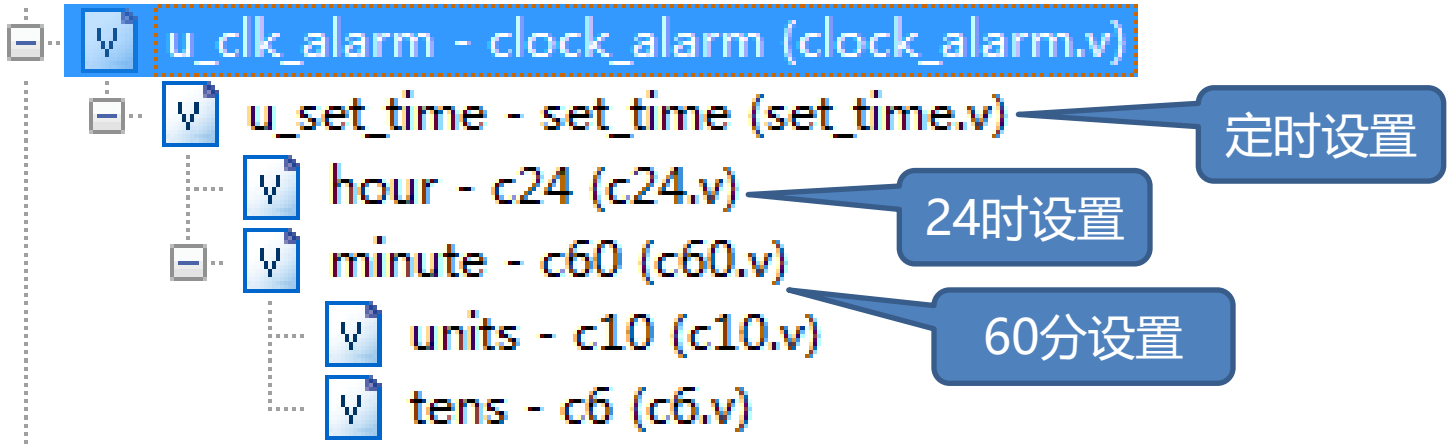
整体代码结构


digital_clock
xc7a100t-3csg324
top (top.v)
u_clock - clk_div (clk_div.v)
u_clk_adj - clock_adjust (clock_adjust.v)
u_clk_alarm - clock_alarm (clock_alarm.v)
u_day24_12 - day24_12 (day24-12.v)
radio_day24_12 - day24_12 (day24-12.v)
u_radio - radio (radio.v)
u_scan_disp - scan_disp (scan_disp.v)
顶层模块
时钟源模块
计时、校时模块
定时、闹钟模块
24/12小时切换模块
仿电台报时
动态显示控制

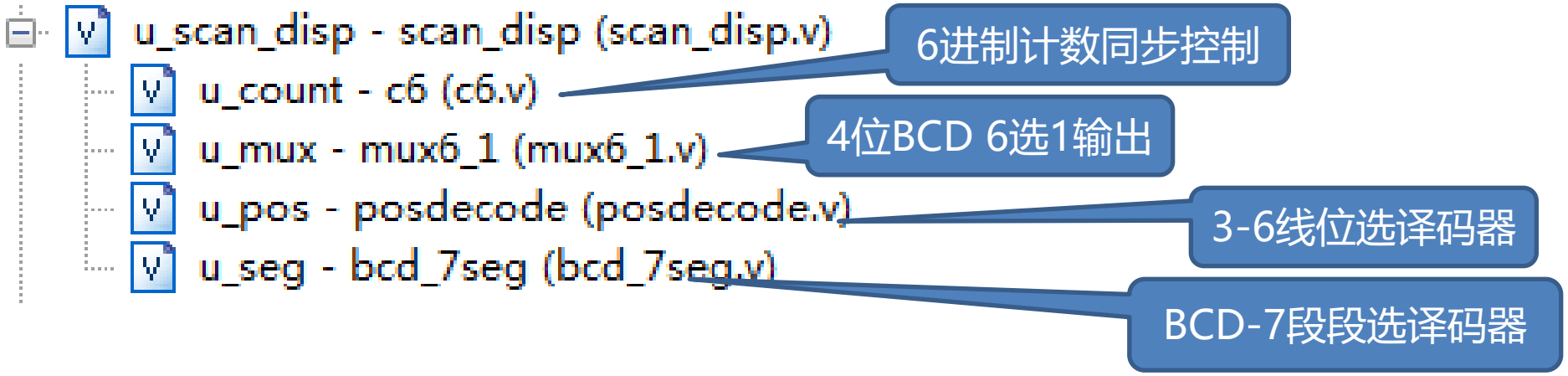
计时、校时模块


u_clk_adj - clock_adjust (clock_adjust.v)
second - c60 (c60.v)
units - c10 (c10.v)
tens - c6 (c6.v)
minute - c60 (c60.v)
units - c10 (c10.v)
tens - c6 (c6.v)
hour - c24 (c24.v)
个位BCD
十进制
十位BCD
六进制
60秒
60分
24时

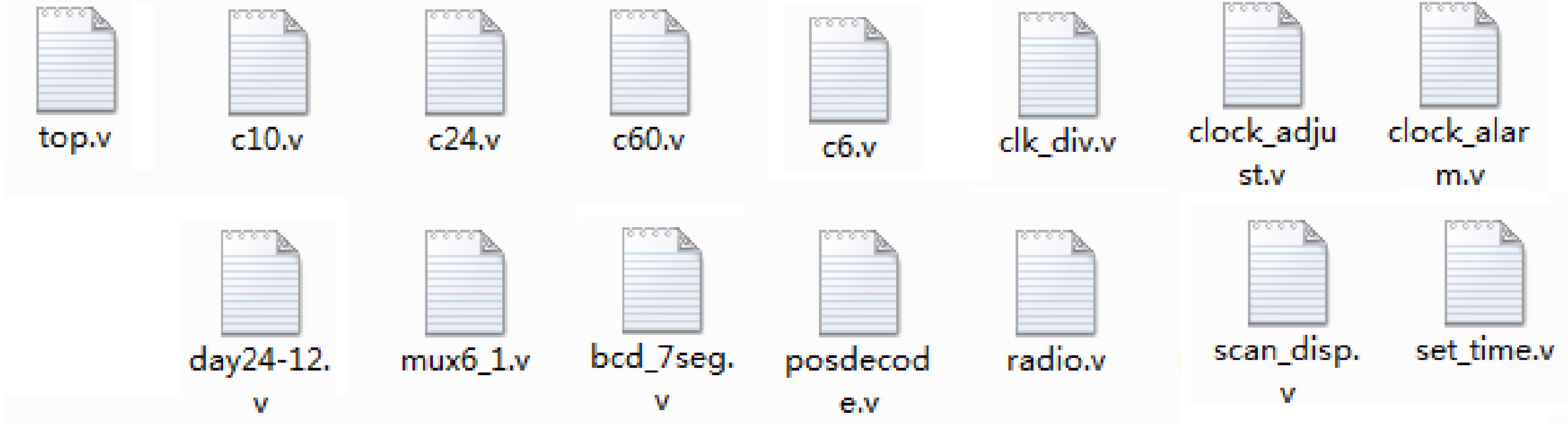
闹钟模块


u_clk_alarm - clock_alarm (clock_alarm.v)
u_set_time - set_time (set_time.v)
hour - c24 (c24.v)
minute - c60 (c60.v)
units - c10 (c10.v)
tens - c6 (c6.v)
定时设置
24时设置
60分设置

动态显示模块


u_scan_disp - scan_disp (scan_disp.v)
u_count - c6 (c6.v)
u_mux - mux6_1 (mux6_1.v)
u_pos - posdecode (posdecode.v)
u_seg - bcd_7seg (bcd_7seg.v)
6进制计数同步控制
4位BCD 6选1输出
3-6线位选译码器
BCD-7段段选译码器

源文件列表


top.v
c10.v
c24.v
c60.v
c6.v
clk_div.v
clock_adjust.v
clock_alarm.v
day24-12.v
mux6_1.v
bcd_7seg.v
posdecode.v
radio.v
scan_disp.v
set_time.v

顶层源代码

```
module top(
    input clk_100m,
    input cr,
    input en_clock,
    input clock_adjust,
    input clock_alarm_en,
    input clock_alarm_set,
    input min_hour_set,
    input alarm_adjust_disp,
    input day_night,
    output [7:0] pos,
    output [6:0] seg,
    output alarm,
    output day
    );
wire clk_1hz,clk_1k,clk_5h,clock_alarm,radio_alarm,mux_sel_set,day_radio;
wire [3:0] bcd_day_hu,bcd_day_ht,bcd_day_ht_radio,bcd_day_hu_radio;
wire [3:0] bcd_smu,bcd_smt,bcd_shu,bcd_sht,bcd_mu,bcd_mt,bcd_su,bcd_st;
wire [3:0] bcd_hu,bcd_ht,bcd_temp_mu,bcd_temp_mt,bcd_temp_hu,bcd_temp_ht;
assign alarm=radio_alarm|clock_alarm&clock_alarm_en;//整点报时或闹铃
assign mux_sel_set=clock_alarm_set|alarm_adjust_disp;//设置闹钟或显示闹钟
assign {bcd_temp_mu,bcd_temp_mt,bcd_temp_hu,bcd_temp_ht}=mux_sel_set?{bcd_smu,bcd_smt,bcd_shu,bcd_sht}:{bcd_mu,bcd_mt,bcd_hu,bcd_ht};//显示2选1
clk_div u_clock(clk_100m,clk_1k,clk_5h,clk_1hz,cr);//分频
clock_adjust u_clk_adj(clk_1hz,clock_adjust,cr,min_hour_set,bcd_su,bcd_st,bcd_mu,bcd_mt,bcd_hu,bcd_ht);//计时、校时
clock_alarm u_clk_alarm(clk_1hz,clk_1k,clk_5h,clock_alarm_en,clock_alarm_set,cr,min_hour_set,bcd_mu,bcd_mt,
bcd_hu,bcd_ht,bcd_smu,bcd_smt,bcd_shu,bcd_sht,clock_alarm);
day24_12 u_day24_12(bcd_temp_ht,bcd_temp_hu,day_night,day,bcd_day_ht,bcd_day_hu);//显示24/12小时调整
day24_12 radio_day24_12(bcd_ht,bcd_hu,day_night,day_radio,bcd_day_ht_radio,bcd_day_hu_radio);//仿电台24/12小时调整
radio u_radio(bcd_su,bcd_st,bcd_mu,bcd_mt,bcd_day_ht_radio,bcd_day_hu_radio,clk_1k,clk_5h,clk_1hz,en_clock,cr,day_night,radio_alarm);
scan_disp u_scan_disp(clk_1k,cr,en_clock,bcd_day_ht,bcd_day_hu,bcd_temp_mt,bcd_temp_mu,bcd_st,bcd_su,seg,pos);
endmodule
```

6进制BCD计数器

```
module c6(
    input clk,
    input cr,
    output rco,
    input en,
    output [3:0] bcd6
    );
reg [3:0] bcd6r;
assign rco=(bcd6r==5)?1'b1:1'b0;//进位信号
assign bcd6=bcd6r ;
always @(posedge clk or negedge cr)//6进制计数
   if (!cr)
      bcd6r <=4'b0000;
      else if (en)
            if (bcd6r!=5)
               bcd6r<=bcd6r+1'b1;
            else
            bcd6r<=0;
         else bcd6r<=bcd6r;

endmodule
```

10进制计数器

```
module c10(
    input clk,
    input en,
    input cr,
    output rco,
    output [3:0] bcd10
    );
reg [3:0] bcd10r;
assign rco=(bcd10r == 9)?1'b1:1'b0;//进位信号
assign bcd10=bcd10r ;
always @(posedge clk or negedge cr)//10进制计数
   if (!cr)
      bcd10r <=0;
      else if (en)
            if (bcd10r<9)
               bcd10r <= bcd10r+1'b1;
            else
            bcd10r<=0;
         else  bcd10r<=bcd10r;

endmodule
```

60进制

```
module c60(
    input clk,
    input en,
    input cr,
    output rco,
    output [3:0] bcd_t,
    output [3:0] bcd_u
    );
wire rco_u,rco_t;
assign rco=rco_u&rco_t; //60进位信号
c10 units(clk,en,cr,rco_u,bcd_u);
c6 tens(clk,cr,rco_t,rco_u&en,bcd_t);
endmodule
```

24进制

```
module c24(
    input clk,
    input en,
    input cr,
    output [3:0] bcd_u,
    output [3:0] bcd_t
    );
reg [3:0] bcd_ur,bcd_tr;
assign bcd_u = bcd_ur;
assign bcd_t = bcd_tr;
always @ (posedge clk or negedge cr)
   if (!cr)
   begin
      bcd_ur<=0;
      bcd_tr<=0;
      end
      else if(en)
         if ((bcd_ur==3)&(bcd_tr==2))//24进制计数
            begin
            bcd_ur<=0;
            bcd_tr<=0;
            end
         else if(bcd_ur==9)
            begin bcd_tr <=bcd_tr+1'b1;
               bcd_ur <= 0;
               end
               else
               bcd_ur <= bcd_ur+1'b1;
endmodule
```

计时、校时模块

```
module clock_adjust(
    input clk,
    input adjust,
    input cr,
    input min_hour,
    output [3:0] bcd_su,
    output [3:0] bcd_st,
    output [3:0] bcd_mu,
    output [3:0] bcd_mt,
    output [3:0] bcd_hu,
    output [3:0] bcd_ht
    );
wire m_en,h_en,rco_s,rco_m;
assign m_en =rco_s|(adjust&min_hour);//分钟计时使能信号
assign h_en=(rco_s&rco_m)|(adjust&~min_hour);//小时计时使能信号
c60 second(clk,1'b1,cr,rco_s,bcd_st,bcd_su);
c60 minute(clk,m_en,cr,rco_m,bcd_mt,bcd_mu);
c24 hour(clk,h_en,cr,bcd_hu,bcd_ht);
```

闹钟设置

```
module set_time(
     input clk,
     input en,
     input min_hour,
     output [3:0] bcd_mu,
     output [3:0] bcd_mt,
     output [3:0] bcd_hu,
     output [3:0] bcd_ht,
     input cr
     );
wire en_min,en_hour,rco60;
assign en_min=en&min_hour;
assign en_hour=en&~min_hour;
c24 hour(clk,en_hour,cr,bcd_hu,bcd_ht);
c60 minute(clk,en_min,cr,rco60,bcd_mt,bcd_mu);

endmodule
```

闹钟铃声控制

```
module clock_alarm(
    input clk_1hz,
    input clk_1k,
    input clk_5h,
    input en_set,
    input cr,
    input min_hour,
    input [3:0] bcd_tmu,
   input [3:0] bcd_tmt,
    input [3:0] bcd_thu,
    input [3:0] bcd_tht,
    output [3:0] bcd_smu,
    output [3:0] bcd_smt,
    output [3:0] bcd_shu,
    output [3:0] bcd_sht,
    output alarm
    );
    wire equ;
    assign equ=(bcd_tmu==bcd_smu)&(bcd_tmt==bcd_smt)&(bcd_thu==bcd_shu)&(bcd_tht==bcd_sht);//比较器
   assign alarm=(equ&clk_1k&clk_1hz)|(equ&clk_5h&~clk_1hz);//1khz，500Hz轮换输出闹铃
    set_time u_set_time(clk_1hz,en_set,min_hour,bcd_smu,bcd_smt,bcd_shu,bcd_sht,cr);//设置时、分闹铃时间

endmodule
```

24/12小时切换

```
module day24_12(
    input [3:0] bcd_ht,
    input [3:0] bcd_hu,
    input day_night,
    output day,
    output [3:0] bcd_hto,
    output [3:0] bcd_huo
    );
wire modify,borrowu,mid_night_zero,noon;
wire [3:0] bcd_temp_hu,bcd_temp_ht,bcd_temp_huo,bcd_temp_hto;
assign noon=(bcd_ht==1)&&(bcd_hu==2);//中午12点
assign modify=(bcd_ht>1)||((bcd_hu>2)&&(bcd_ht==1));//12点以后
assign borrowu=(bcd_hu<2)?1:0;//BCD减法调整
assign bcd_temp_hu=borrowu?(bcd_hu+4'b1010-4'b0010):(bcd_hu-4'b0010);
assign bcd_temp_ht=borrowu?(bcd_ht-4'b0010):(bcd_ht-4'b0001);
assign mid_night_zero = (bcd_ht==0)&&(bcd_hu==0);//零点
assign bcd_temp_huo=(modify&day_night)?bcd_temp_hu:bcd_hu;
assign bcd_temp_hto=(modify&day_night)?bcd_temp_ht:bcd_ht;
assign {bcd_hto,bcd_huo} =(mid_night_zero&day_night)?8'h12:{bcd_temp_hto,bcd_temp_huo};//午夜12点
assign day=(modify|noon)&day_night;//12点以后标志下午
endmodule
```

仿电台报时

```
module radio(
    input [3:0] bcd_su,
    input [3:0] bcd_st,
    input [3:0] bcd_mu,
    input [3:0] bcd_mt,
    input [3:0] bcd_hu,
    input [3:0] bcd_ht,
    input clk_1k,
    input clk_5h,
    input clk_1hz,
    input en,
    input cr,
    input day_night,
    output radio_alarm
    );
wire ld,min_equ;
wire equ,grt,less,noon;
wire [3:0] bcd_temp_hu,bcd_temp_ht;
assign noon= ({bcd_ht,bcd_hu}==8'h12);//中午12点标志
assign {bcd_temp_ht,bcd_temp_hu}=(noon&day_night)?8'h00:{bcd_ht,bcd_hu};//中午12点且12小时制则是下午0点
assign min_equ=((bcd_mt==5)&&(bcd_mu==9))?1'b1:1'b0;//整点来临前
assign equ =({bcd_st,bcd_su,bcd_mt,bcd_mu}==16'h0000)?1'b1:1'b0;//整点
assign grt =(({bcd_st,bcd_su}+{bcd_temp_ht,bcd_temp_hu}+8'h1)>8'h5a)?1'b1:1'b0;//提示整点来临前的秒数
assign radio_alarm=(((grt&min_equ&clk_5h)|(equ&clk_1k))&clk_1hz)&en;//整点来临前输出500HZ信号，整点时输出1kHz信号

endmodule
```

6选1复用器

```
module mux6_1(
      input [3:0] ch0,
      input [3:0] ch1,
      input [3:0] ch2,
      input [3:0] ch3,
      input [3:0] ch4,
      input [3:0] ch5,
      input [3:0] sel,
      output [3:0] bcd
      );
      reg [3:0] bcdr;
      assign bcd = bcdr;
always @(sel)
case (sel)
4'b0000: bcdr <= ch0;
4'b0001: bcdr <= ch1;
4'b0010: bcdr <= ch2;
4'b0011: bcdr <= ch3;
4'b0100: bcdr <= ch4;
4'b0101: bcdr <= ch5;
default: bcdr <= 4'b1111;
endcase
```

BCD-7段数码管译码

```
module bcd_7seg(
    input [3:0] bcd,
    output [6:0] seg
    );
    reg [6:0] seg1;
    assign seg =seg1;
always @(bcd)
 case (bcd)
            4'b0000  : seg1 <= 7'b1000000;//40
            4'b0001  : seg1 <= 7'b1111001;//79
            4'b0010  : seg1 <= 7'b0100100;//24
            4'b0011  : seg1 <= 7'b0110000;//30
            4'b0100  : seg1 <= 7'b0011001;//19
            4'b0101  : seg1 <= 7'b0010010;//12
            4'b0110  : seg1 <= 7'b0000010;//2
            4'b0111  : seg1 <= 7'b1111000;//78
            4'b1000  : seg1 <= 7'b0000000;//0
            4'b1001  : seg1 <= 7'b0010000; //10
            default : seg1 <= 7'b1111111;
        endcase
```

位选译码

```
module posdecode(
    input [3:0] bcd6,
    output [7:0] pos
    );
    reg [7:0] posr;
    assign pos = posr;
always @(bcd6)
case (bcd6)
4'b0000: posr <= 8'b11011111;
4'b0001: posr <= 8'b11101111;
4'b0010: posr <= 8'b11110111;
4'b0011: posr <= 8'b11111011;
4'b0100: posr <= 8'b11111101;
4'b0101: posr <= 8'b11111110;
default: posr <= 8'b11111111;
endcase

endmodule
```

动态显示控制

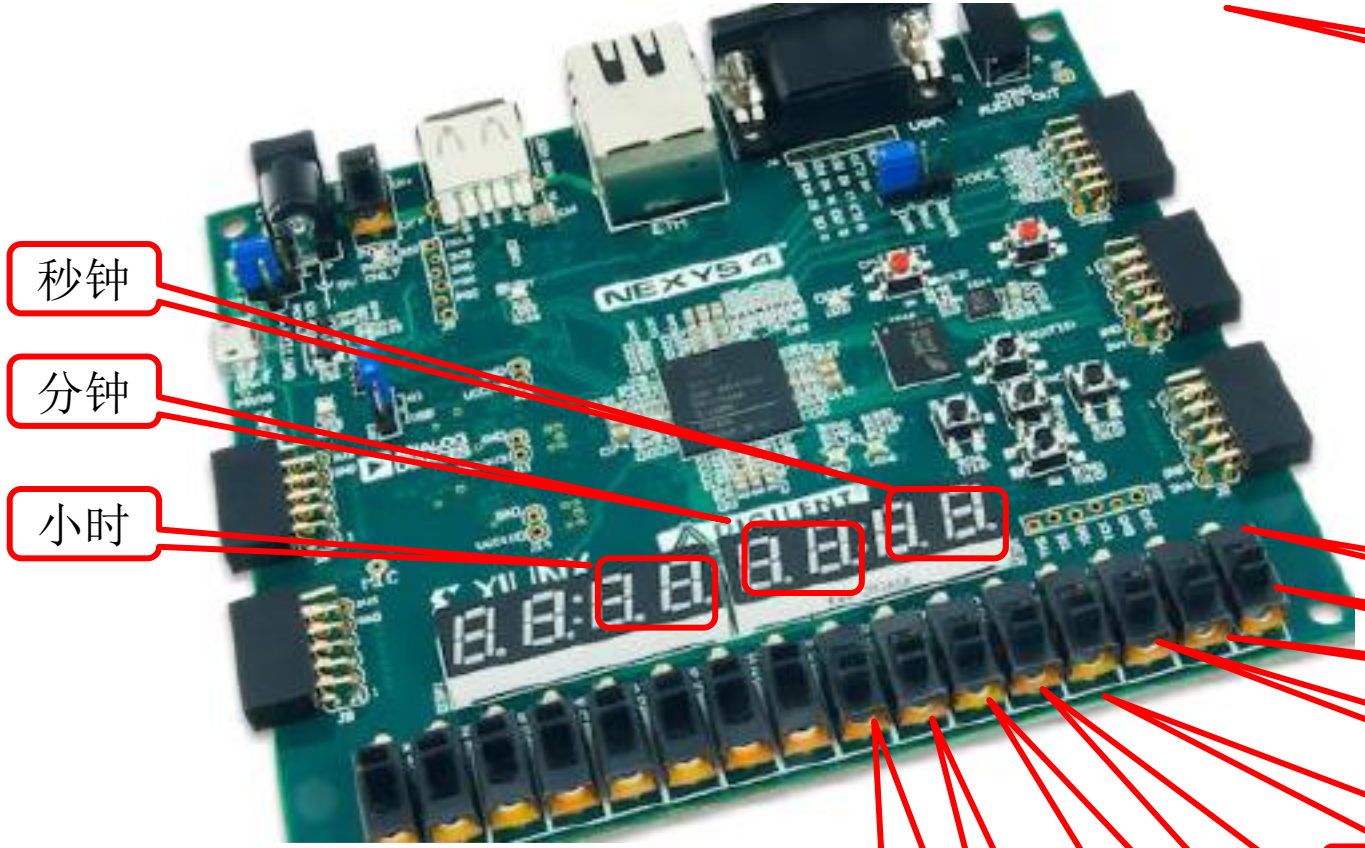
```
module scan_disp(
    input clk,
    input cr,
    input en,
    input [3:0] ch0,
    input [3:0] ch1,
    input [3:0] ch2,
    input [3:0] ch3,
    input [3:0] ch4,
    input [3:0] ch5,
    output [6:0] seg,
    output [7:0] pos
    );
    wire rco;
    wire [3:0] bcdsel,bcd_data;
   c6 u_count(clk,cr,rco,en,bcdsel);//6进制同步计数器
   mux6_1 u_mux(ch0,ch1,ch2,ch3,ch4,ch5,bcdsel,bcd_data);//6路4位复用器
   posdecode u_pos(bcdsel,pos);//位码
   bcd_7seg u_seg(bcd_data,seg);//段码

endmodule
```

计时、校时模块仿真

```
parameter PERIOD = 10;
    always begin
        clk = 1'b0;
        #(PERIOD/2) clk = 1'b1;
        #(PERIOD/2);
    end
    initial begin
        // Initialize Inputs
        clk = 0;
        adjust = 0;
        cr = 0;
        min_hour = 0;
        // Wait 100 ns for global reset to finish
        #10;
        adjust = 0;
        cr = 1;
        min_hour = 0;
        #100000;//正常计时100000ns
        adjust = 1;
        min_hour = 0;
        #100;//校时100ns
        min_hour = 1;
        #100;//校分100ns
        adjust = 0;//恢复正常计时
        // Add stimulus here
    end
```

约束定义



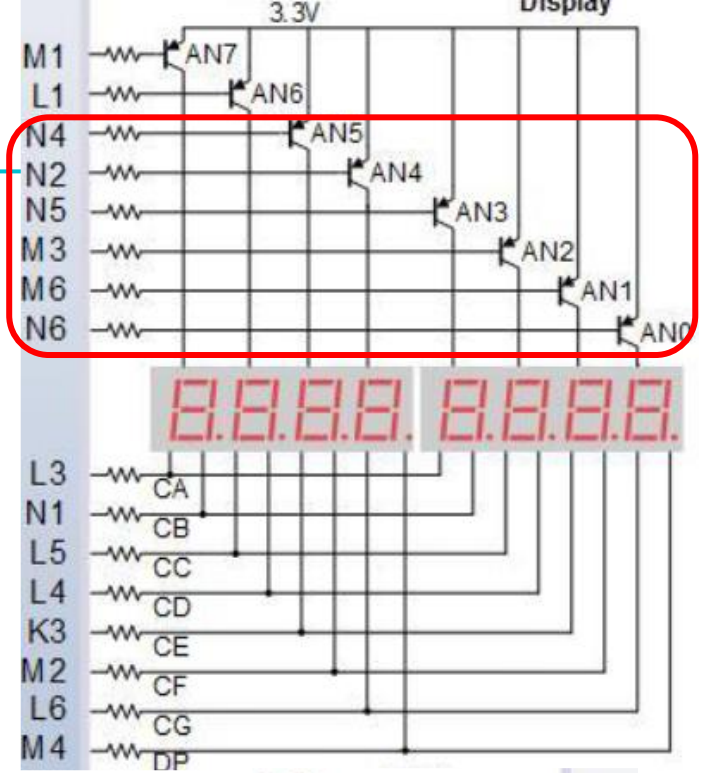
秒钟

分钟

小时

T8 LD0

AM/PM



AM/PM

en_clock

clock_adjust

clock_alarm_en

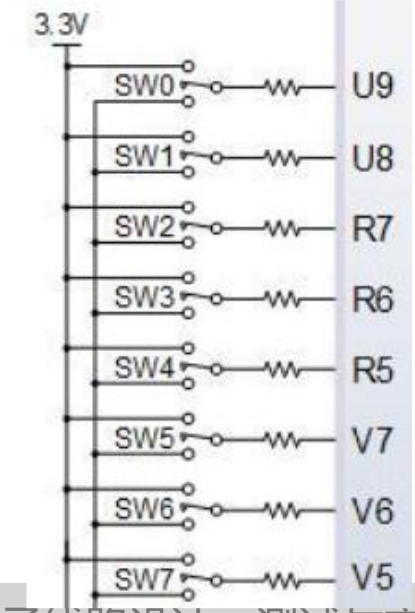
clock_alarm_set

min_hour_set

cr

day_night

Alarm_adjust_disp



谢谢观看

華中科技大學
Huazhong University of Science and Technology